

Laboratorio 3: Protocolos de Enrutamiento

Esteban Cárcamo (23016)
Jorge Luis Felipe Aguilar (23195)
13 de agosto de 2026

1. Descripción de la práctica

La práctica consistió en implementar el protocolo de enrutamiento **Link State** sobre una red simulada de nueve nodos que se comunican mediante sockets TCP. Cada nodo es un proceso independiente que arranca conociendo únicamente a sus vecinos directos y el costo de cada enlace; el resto de la topología lo descubre intercambiando mensajes con la red.

El trabajo se dividió en dos planos que corren en paralelo dentro de cada nodo. El **plano de control** descubre vecinos con paquetes HELLO, construye el anuncio de estado de enlace (LSA), lo propaga por inundación, arma el grafo completo de la red y calcula las rutas más cortas con el algoritmo de Dijkstra, dejando el resultado en un archivo *nodo_tabla_enrutamiento.csv*. El **plano de datos** consulta esa tabla para reenviar los mensajes de aplicación salto por salto, aplicando detección y corrección de errores con código Hamming(7,4).

Sobre esa red viaja el tráfico entre dos equipos terminales que no son routers: un cajero automático (ATM) y un servidor bancario, cada uno conectado a su router de puerta de enlace predeterminada. La topología y el formato de los mensajes se acordaron con las otras dos parejas del grupo para garantizar interoperabilidad entre las tres implementaciones, y las pruebas de interconexión se realizaron sobre la red privada virtual Tailscale.

2. Descripción de los algoritmos utilizados y su implementación

2.1 Arquitectura general

Cada nodo escucha en un único puerto TCP. Al recibir una línea, el despachador la clasifica por contenido: si comienza con `{` es un mensaje de control en JSON, y si contiene solo los caracteres `0` y `1` es una trama Hamming del plano de datos. Ambas formas son excluyentes, de modo que la regla no presenta casos ambiguos. Los dos planos se ejecutan en hilos independientes y su único punto de contacto es el archivo CSV de la tabla de enrutamiento: el plano de control lo escribe y el plano de datos lo lee.

2.2 Descubrimiento de vecinos: HELLO

Cada nodo envía un paquete HELLO cada 5 segundos a todos sus vecinos configurados, estén activos o no, ya que ese es el mecanismo por el cual se descubren mutuamente. Un vecino se declara caído tras 15 segundos sin recibir su HELLO, y en ese instante el nodo emite un LSA

propio con el número de secuencia incrementado y sin ese enlace. El HELLO lleva $ttl = 1$ porque nace y muere en el enlace: nunca se reenvía. El costo del enlace no se mide, se lee de la topología compartida y es simétrico.

2.3 Anuncio de estado de enlace: LSA

El LSA contiene exclusivamente los enlaces directos activos del nodo que lo origina, nunca rutas calculadas. Su estructura es la siguiente:

```
{ "proto": "LinkState", "type": "LSA", "origin": "A", "seq": 3,
  "links": { "B": 7, "I": 1, "C": 7 }, "from": "A", "ttl": 8 }
```

El campo *origin* identifica al dueño del anuncio y no cambia nunca; *from* indica el vecino que lo reenvió y se modifica en cada salto. El número de secuencia *seq* permite distinguir anuncios nuevos de anuncios viejos y se persiste en disco, de manera que un nodo que reinicia no vuelve a numerar desde cero. Si aun así la red recuerda un LSA propio con secuencia mayor, el contador la supera y se reanuncia de inmediato.

Se emite un LSA al arrancar, cuando aparece un vecino nuevo, cuando se cae un vecino y cada 30 segundos como refresco periódico. Este refresco permite que un nodo que se incorpora tarde complete su grafo sin depender de un cambio de topología.

2.4 Inundación controlada (flooding)

Al recibir un LSA ajeno, el nodo aplica tres reglas en orden. Primero, si el número de secuencia es menor o igual al mayor visto para ese origen, el anuncio se descarta sin reenviar; aquí es donde muere el ciclo y donde se evita la tormenta de difusión. Segundo, si es mayor, se almacena en la base de datos de estado de enlaces y se reenvía a todos los vecinos activos excepto a aquel que lo envió, técnica conocida como *split horizon*. Tercero, antes de reenviar se decrementa el *ttl*, que inicia en 8, y el paquete se descarta al llegar a cero; esto actúa como respaldo ante un nodo que reinicie con una secuencia baja.

Como optimización de convergencia, cuando llega el primer HELLO de un vecino se le envían de inmediato todos los anuncios almacenados. Son LSA normales, por lo que cualquier implementación del grupo los procesa sin cambios, pero reducen sustancialmente el tiempo de convergencia.

2.5 Construcción del grafo

Un enlace ingresa al grafo únicamente si **ambos extremos lo anuncian**. Esta decisión resuelve un problema concreto: el protocolo acordado no define envejecimiento de los LSA, por lo que el anuncio de un nodo caído permanecería almacenado indefinidamente en los demás. Al exigir reciprocidad, los vecinos activos del nodo caído lo desconectan mediante su propio LSA actualizado y el nodo muerto queda fuera de todo camino sin necesidad de un mecanismo de expiración. Cuando los costos anunciados por cada extremo difieren, se toma el mayor por ser la lectura conservadora.

2.6 Cálculo de rutas: Dijkstra

Cada vez que el grafo conocido cambia, se ejecuta el algoritmo de Dijkstra desde el nodo local. El resultado no es la ruta completa sino, para cada destino alcanzable, el **primer salto** y

el costo acumulado. Los destinos inalcanzables simplemente no aparecen en la tabla. El desempate entre rutas de igual costo es estable y determinista, lo que garantiza que dos ejecuciones con el mismo grafo produzcan tablas idénticas.

La tabla se escribe de forma atómica en *nodo_tabla_enrutamiento.csv* con el formato acordado:

```
destino,siguiente_salto,costo,ip,puerto
```

Es importante notar que los campos *ip* y *puerto* corresponden al **siguiente salto** y no al destino final, ya que es a esa dirección a la que el plano de datos abre el socket.

2.7 Plano de datos y arquitectura de capas

El reenvío implementa el ciclo completo descrito en el enunciado: se recibe una cadena de bits, se detectan y corrigen errores sobre todos los bits, se extraen los bits de datos, se deserializa y se lee el destino, se consulta la tabla de ruteo, se abre la conexión según la IP y el puerto devueltos, se serializa nuevamente, se aplica Hamming(7,4) y se envían los bits por el socket.

El sobre de datos tiene la siguiente estructura:

```
{ "type": "message", "from": "A", "to": "E", "ttl": 16,  
  "hops": ["A"], "payload": { "op": "withdraw", "amount": 500 } }
```

Los routers leen exclusivamente el campo *to*. El *payload* nunca se abre, pues corresponde al diálogo entre el ATM y el banco y ningún nodo intermedio debe interpretarlo; esta es la separación de capas que exige la práctica. Antes de reenviar, el nodo se agrega a la lista *hops* y decrementa el *ttl*. El mensaje se descarta si el *ttl* llega a cero o si el nodo se encuentra a sí mismo en *hops*, lo que indicaría un bucle de ruteo. Cuando *to* coincide con el nodo local, el mensaje no se reenvía sino que se entrega al equipo terminal conectado a ese router.

2.8 Detección y corrección de errores: Hamming(7,4)

El sobre serializado se convierte a bytes UTF-8 y cada byte se divide en dos nibbles, empezando por el más significativo. Cada nibble *d1 d2 d3 d4* produce un bloque de siete bits con el orden *p1 p2 d1 p3 d2 d3 d4* y paridad par:

$p1 = d1 \wedge d2 \wedge d4$	cubre las posiciones 1, 3, 5, 7
$p2 = d1 \wedge d3 \wedge d4$	cubre las posiciones 2, 3, 6, 7
$p3 = d2 \wedge d3 \wedge d4$	cubre las posiciones 4, 5, 6, 7

Resultan 14 bits exactos por byte, sin relleno ni metadatos, de modo que el receptor reconstruye el mensaje únicamente a partir de la cadena de bits. Al decodificar, el síndrome indica la posición del bit a corregir, o cero si el bloque llegó intacto. El código corrige un bit por bloque de siete; con dos o más errores en el mismo bloque la corrección converge a un valor equivocado, situación que se detecta al fallar la decodificación UTF-8 o el análisis del JSON. Esta capa se aplica únicamente al plano de datos; los mensajes de control viajan como JSON plano.

2.9 Equipos terminales

El ATM y el servidor bancario no son routers: no participan del plano de control, no poseen tabla de enrutamiento y no ejecutan Link State. Cada uno conoce únicamente su puerta de enlace predeterminada, que es el router que lo declara como equipo terminal. Ambos escuchan además en su propio puerto, porque la conexión de ida y la de retorno son independientes y cada extremo abre la suya para escribir. El rol de cada nodo puede declararse por línea de comandos, lo que permite reubicar el ATM o el banco en cualquier router sin modificar el archivo de direcciones compartido con las otras parejas.

3. Resultados

3.1 Convergencia de la topología

Se ejecutaron los nueve routers como procesos independientes sobre la topología acordada de nueve nodos y once enlaces. La red conocida por cada nodo alcanzó los **9 nodos y 11 enlaces** en aproximadamente **7 segundos** desde el arranque, con el grafo siguiente:

A-B=7 A-C=7 A-I=1 B-F=2 C-D=5 D-E=1
D-F=1 D-I=6 E-G=4 F-G=3 F-H=4

3.2 Transmisión extremo a extremo

Con la topología convergida se ejecutaron transacciones completas entre el ATM y el servidor bancario. Se probó además la reubicación de los equipos terminales para verificar que el enrutamiento es independiente de su posición. La cuenta de prueba 23016 partió de un saldo de \$400.00 en cada corrida.

Configuración	Ruta observada	Costo	Resultado
ATM en A, banco en E	A → I → D → E	8	Autenticación, retiro de \$50.00 y cierre de sesión
ATM en H, banco en E	H → F → D → E	6	Autenticación, retiro de \$50.00 y cierre de sesión
ATM en A, banco en H	A → I → D → F → H	12	Autenticación, retiro de \$75.00 y cierre de sesión

La bitácora del primer caso registró la ruta de ida y su retorno por el camino inverso:

```
A: mensaje hacia E reenviado por I; ruta=['A']
I: mensaje hacia E reenviado por D; ruta=['A', 'I']
D: mensaje hacia E reenviado por E; ruta=['A', 'I', 'D']
E: mensaje entregado al host local; ruta=['A', 'I', 'D', 'E']

E: mensaje hacia A reenviado por D; ruta=['E']
D: mensaje hacia A reenviado por I; ruta=['E', 'D']
I: mensaje hacia A reenviado por A; ruta=['E', 'D', 'I']
A: mensaje entregado al host local; ruta=['E', 'D', 'I', 'A']
```

3.3 Tabla de enrutamiento generada

La tabla calculada por el nodo A confirma que el primer salto hacia E es I, y no el enlace directo A–C que a simple vista parecería más corto:

Destino	Siguiente salto	Costo
B	B	7
C	C	7
D	I	7
E	I	8
F	I	8
G	I	11
H	I	12
I	I	1

3.4 Pruebas de interconexión sobre Tailscale

Se configuró la red privada virtual Tailscale con los seis integrantes de las tres parejas y se reemplazaron las direcciones locales por las direcciones asignadas en el rango *100.x.y.z*. Los nodos se distribuyeron entre cinco máquinas de acuerdo con el reparto acordado.

En la sesión de prueba se logró interoperabilidad entre las implementaciones de las tres parejas para los nodos que estuvieron activos simultáneamente. El nodo local recibió correctamente los HELLO de su vecino y procesó los LSA originados por nodos de las otras dos implementaciones, alcanzando un grafo de **6 nodos y 5 enlaces** (A–B, A–I, B–F, F–G, F–H). Los tres nodos restantes no estuvieron disponibles durante la ventana de prueba, por lo que la convergencia total sobre Tailscale quedó pendiente de una sesión con los seis integrantes conectados de forma simultánea.

3.5 Pruebas automatizadas

La implementación cuenta con **204 pruebas unitarias** que finalizaron satisfactoriamente. Cubren la topología real calculada manualmente, el desempate entre rutas de igual costo, los destinos inalcanzables, la persistencia del archivo CSV, el descarte por TTL agotado, la detección de bucles, la codificación y corrección Hamming, y las operaciones del servidor bancario.

4. Discusión

Los resultados comprueban que el plano de control y el de datos operan en paralelo y se comunican exclusivamente mediante la tabla CSV. La escritura atómica de dicho archivo evita lecturas parciales mientras el grafo reconverge, situación que ocurre con frecuencia durante el arranque escalonado de los nodos.

La ruta observada coincide con el resultado teórico de Dijkstra y demuestra que el tráfico no sigue el enlace que aparenta ser más directo. En el caso de A hacia E, la ruta por C costaría 13 mientras que la ruta $A \rightarrow I \rightarrow D \rightarrow E$ cuesta 8. De igual forma, al ubicar el banco en H, el tráfico eligió $A \rightarrow I \rightarrow D \rightarrow F \rightarrow H$ con costo 12 en lugar de $A \rightarrow B \rightarrow F \rightarrow H$ con costo 13. Esto confirma que las tablas reflejan decisiones basadas en el costo acumulado y no en el número de saltos.

El TTL y la lista *hops* aportan defensas complementarias contra bucles: el primero limita cualquier circulación indefinida y la segunda detecta de inmediato el retorno a un router ya visitado. Ninguna de las dos resultó suficiente por sí sola durante el desarrollo, pues el TTL permite que un mensaje circule varias veces antes de expirar mientras que *hops* no protege contra un mensaje malformado que llegue con la lista vacía.

La decisión de admitir en el grafo únicamente los enlaces anunciados por ambos extremos resultó determinante durante las pruebas sobre Tailscale. Al no existir envejecimiento de los LSA en el protocolo acordado, los anuncios de nodos que se desconectaron permanecían en la base de datos; la exigencia de reciprocidad hizo que esos nodos quedaran automáticamente excluidos de todo camino en cuanto sus vecinos activos actualizaron sus propios anuncios. Este comportamiento se observó directamente en la bitácora, donde un nodo pasó de anunciar tres enlaces a anunciar dos tras el vencimiento del temporizador de vecino.

La principal limitación encontrada no fue de naturaleza técnica sino de coordinación. La convergencia completa sobre Tailscale exige que los nueve procesos estén activos dentro de la misma ventana de 15 segundos que dura el temporizador de vecino; de lo contrario, los nodos que arrancan tarde son declarados caídos por quienes ya estaban en línea. Se comprobó que la sincronización de la base de datos al detectar un vecino nuevo mitiga parcialmente este problema, pues acorta la convergencia de 30 segundos a aproximadamente 7.

Respecto a la interoperabilidad, el hecho de que los LSA generados por las tres implementaciones se procesaran sin modificaciones confirma que la definición del protocolo fue suficientemente precisa. Las únicas ambigüedades detectadas fueron los costos de los enlaces B–F, D–E y F–G, difíciles de leer en el diagrama del enunciado, y el formato exacto de las respuestas exitosas del banco. Ambas se resolvieron por acuerdo explícito entre las parejas antes de las pruebas.

5. Conclusiones

El protocolo Link State se implementó de forma completa y funcional: el descubrimiento de vecinos mediante HELLO, la construcción y propagación de los LSA por inundación controlada, el armado del grafo de la red y el cálculo de rutas mínimas con Dijkstra operan de manera autónoma en cada nodo sin conocimiento previo de la topología global.

La separación entre plano de control y plano de datos demostró ser correcta en la práctica. Ambos planos se ejecutan concurrentemente y su único acoplamiento es la tabla de

enrutamiento, lo que permitió reubicar los equipos terminales sin modificar una sola línea del plano de control.

La inundación controlada mediante números de secuencia es suficiente para evitar tormentas de difusión. El descarte de anuncios con secuencia menor o igual a la ya conocida detiene la propagación en un número acotado de saltos, y el TTL actúa como respaldo ante nodos que reinician.

La arquitectura de capas se cumplió estrictamente: ningún router intermedio interpretó el contenido de los mensajes de aplicación, limitándose a leer el campo de destino y consultar su tabla. Esta propiedad es la que permite que las tres implementaciones interoperen pese a haber sido desarrolladas de forma independiente.

El código Hamming(7,4) cumple su función de corrección de un bit por bloque, aunque su capacidad es limitada frente a errores en ráfaga. En un medio confiable como TCP sobre Tailscale su aporte práctico es reducido, pero cumple el objetivo didáctico de ejercitar la capa de detección y corrección de errores dentro de la pila.

6. Comentarios

La dificultad principal del laboratorio no residió en los algoritmos sino en la coordinación entre las tres parejas. Acordar de antemano los costos, los identificadores de nodo y el formato exacto de los mensajes resultó ser más determinante para el éxito que cualquier detalle de implementación.

Se recomienda para futuras iteraciones incorporar un mecanismo de envejecimiento de los LSA, de modo que los anuncios de nodos desconectados expiren por sí solos en lugar de depender de la regla de reciprocidad. Asimismo, sería conveniente definir un procedimiento de arranque coordinado que evite que los nodos tempranos declaren caídos a los tardíos.

7. Referencias

Kurose, J. F., & Ross, K. W. (2021). *Computer Networking: A Top-Down Approach* (8th ed.). Pearson.

Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5th ed.). Prentice Hall.

Moy, J. (1998). *OSPF Version 2* (RFC 2328). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc2328>

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271.

Hamming, R. W. (1950). Error detecting and error correcting codes. *The Bell System Technical Journal*, 29(2), 147–160.

Tailscale Inc. (2026). *Tailscale Documentation*. <https://tailscale.com/kb>